

Internal-Lab Exam

NAME – P.Laxminivas

ROLL NO – 2311CS040141

SUB – AGENTIC AI

6. Policy Compliance Agent

Build an agent with rule-based evaluation and synthetic data

Code:

```
import re
import random
import pandas as pd
import matplotlib.pyplot as plt
from faker import Faker

fake = Faker()
Faker.seed(42)
random.seed(42)

TOXIC_WORDS = ["idiot", "stupid", "dumb", "hate", "moron"]

def check_policy(text: str) -> dict:
```

```
"""
```

Evaluates a string against policy rules.

Returns a dictionary with compliance status and violation details.

```
"""
```

```
violations = []
```

```
ssn_pattern = re.compile(r"\b\d{3}-\d{2}-\d{4}\b")
```

```
if ssn_pattern.search(text):
```

```
    violations.append("PII_DETECTED_SSN")
```

```
text_lower = text.lower()
```

```
if any(word in text_lower for word in TOXIC_WORDS):
```

```
    violations.append("TOXICITY_DETECTED")
```

```
is_compliant = len(violations) == 0
```

```
return {
```

```
    "is_compliant": is_compliant,
```

```
    "violations": violations
```

```
}
```

```
def generate_synthetic_data(num_samples=100):
```

```
    data = []
```

```
    for _ in range(num_samples):
```

```
        msg_type = random.choices(["safe", "pii", "toxic", "both"], weights=[0.6, 0.2, 0.15, 0.05])[0]
```

```
        if msg_type == "safe":
```

```
            text = fake.sentence()
```

```
        elif msg_type == "pii":
```

```
            text = f"My application number is {fake.ssn()} please check."
```

```
        elif msg_type == "toxic":
```

```
            word = random.choice(TOXIC_WORDS)
```

```

    word = random.choice(TOXIC_WORDS)

    text = f"You {word}, my SSN is {fake.ssn()}!"

    data.append({"text": text, "intended_type": msg_type})

return data

def main():

    print("Generating synthetic data...")

    dataset = generate_synthetic_data(100)

    print("Evaluating data against policies...")

    results = []

    for item in dataset:

        eval_result = check_policy(item["text"])

        results.append({

            "text": item["text"],

            "intended_type": item["intended_type"],

            "is_compliant": eval_result["is_compliant"],

            "violations": ", ".join(eval_result["violations"]) if eval_result["violations"] else "None"

        })

    df = pd.DataFrame(results)

    total = len(df)

    compliant_count = df["is_compliant"].sum()

    non_compliant_count = total - compliant_count

    print(f"Total messages evaluated: {total}")

    print(f"Compliant: {compliant_count}")

    print(f"Non-compliant: {non_compliant_count}")

```

output:

```
Fine-Tuning Output (Representative Example)

PS C:\AI-Lab> python finetune_llm.py
Loading distilgpt2...
Creating synthetic dataset for domain adaptation...
Configuring Trainer...
Starting fine-tuning...
[Training logs: loss decreases over training steps]
Generating training loss plot...

Testing the fine-tuned model...
Prompt: Patient exhibits symptoms of
Generated: Patient exhibits symptoms of acute bronchitis, recommend amoxicillin.

Fine-tuning complete. Results saved to training_loss_curve.png
```

10. Fine-Tuning for Domain Adaptation

Train and evaluate a specialized LLM

Code:

```
import ollama

import sqlite3

conn = sqlite3.connect("database.db")

cursor = conn.cursor()

cursor.execute("""

CREATE TABLE IF NOT EXISTS students (

    id INTEGER PRIMARY KEY,

    name TEXT,
```

```
        department TEXT,  
        marks INTEGER  
    )  
    """)  
cursor.execute("DELETE FROM students")
```

```
students = [  
    (1, "Rahul", "CSE", 85),  
    (2, "Priya", "ECE", 92),  
    (3, "Arjun", "CSE", 78),  
    (4, "Sneha", "IT", 88),  
    (5, "Kiran", "CSE", 95)  
]
```

```
cursor.executemany(  
    "INSERT INTO students VALUES (?, ?, ?, ?)",  
    students
```

```
conn.commit()
```

```
def execute_sql(sql):
```

```
    try:
```

```
        cursor.execute(sql)
```

```
        result = cursor.fetchall()
```

```
        return result
```

```
    except Exception as e:
```

```
        return f"SQL Error: {e}"
```

```
def sql_agent(question):
```

```
prompt = f"""
```

You are a SQL assistant.

Database table:

```
students(  
    id INTEGER,  
    name TEXT,  
    department TEXT,  
    marks INTEGER  
)
```

Convert the user's question into ONLY a valid SQLite SQL query.

Do not explain.

Do not use markdown.

Return only SQL.

User question:

```
{question}
```

```
"""
```

```
response = ollama.chat(  
    model="llama3.2",  
    messages=[  
        {  
            "role": "user",  
            "content": prompt,  
        },  
    ],  
)
```

```
    model="llama3.2",
```

```
    messages=[
```

```
        {
```

```
            "role": "user",
```

```

        "content": prompt
    }
]
)

sql = response["message"]["content"].strip()

# Remove markdown if model adds it
sql = sql.replace("`sql", "")
sql = sql.replace("```", "")
sql = sql.strip()

print("\nGenerated SQL:")
print(sql)

# Tool use
result = execute_sql(sql)

return result

while True:

    question = input("\nAsk a question (or type exit): ")

    if question.lower() == "exit":
        break

    result = sql_agent(question)

```

```
print("\nTool Result:")

print(result)

import torch

from transformers import AutoModelForCausalLM, AutoTokenizer, TrainingArguments, Trainer

from datasets import Dataset

import matplotlib.pyplot as plt

import os

import pandas as pd
```

output:

```
Policy Agent Output

PS C:\AI-Lab> python policy_agent.py
Generating synthetic data...
Evaluating data against policies...
Total messages evaluated: 100
Compliant: 65
Non-compliant: 35

Evaluation complete. Results saved to policy_evaluation_results.png
and policy_evaluation_log.csv
```

4. SQL Agent with Tool Use

Develop a ReAct-based agent using database tools

Code:

```
import ollama

import sqlite3
```

```
# -----  
  
# Create database  
  
# -----  
  
conn = sqlite3.connect("students.db")  
cursor = conn.cursor()  
  
cursor.execute("""  
CREATE TABLE IF NOT EXISTS students (  
    id INTEGER PRIMARY KEY,  
    name TEXT,  
    department TEXT,  
    marks INTEGER  
)  
""")  
  
# Clear old data  
cursor.execute("DELETE FROM students")  
  
# Insert sample data  
students = [  
    (1, "Rahul", "CSE", 85),  
    (2, "Priya", "ECE", 92),  
    (3, "Arjun", "CSE", 78),  
    (4, "Sneha", "IT", 88),  
    (5, "Kiran", "CSE", 95)  
]
```

```
cursor.executemany(  
    "INSERT INTO students VALUES (?, ?, ?, ?)",  
    students  
)
```

```
conn.commit()
```

```
# -----
```

```
# Text-to-SQL function
```

```
# -----
```

```
def text_to_sql(question):
```

```
    prompt = f"""
```

```
You are a Text-to-SQL system.
```

```
Database table:
```

```
students(  
    id INTEGER,  
    name TEXT,  
    department TEXT,  
    marks INTEGER  
)
```

```
Convert the following natural language question  
into a valid SQLite SQL query.
```

Return ONLY the SQL query.

Do not explain.

Do not use markdown.

Question:

{question}

"""

```
response = ollama.chat(
    model="llama3.2",
    messages=[
        {
            "role": "user",
            "content": prompt
        }
    ]
)

sql = response["message"]["content"].strip()

# Remove markdown if generated
sql = sql.replace("```sql", "")
sql = sql.replace("```", "")
sql = sql.strip()

return sql
```

```
# -----  
  
# Main program  
  
# -----  
  
print("=" * 50)  
print("    TEXT-TO-SQL WORKFLOW")  
print("=" * 50)  
  
while True:  
  
    question = input("\nEnter your question (or type exit): ")  
  
    if question.lower() == "exit":  
        break  
  
    # Generate SQL  
    sql = text_to_sql(question)  
  
    print("\nGenerated SQL:")  
    print(sql)  
  
    # Execute SQL  
    try:  
        cursor.execute(sql)  
        result = cursor.fetchall()  
  
        print("\nQuery Result:")  
        for row in result:
```

```
print(row)
```

```
except Exception as e:
```

```
print("\nSQL Error:", e)
```

```
conn.close()
```

output:

```
PS C:\Users\matam\OneDrive\Desktop\labexam> cd question-01
PS C:\Users\matam\OneDrive\Desktop\labexam\question-01> python text_to_sql.py
=====
TEXT-TO-SQL WORKFLOW
=====

Enter your question (or type exit): TEXT-TO-SQL WORKFLOW

Generated SQL:
SELECT * FROM students;

Query Result:
(1, 'Rahul', 'CSE', 85)
(2, 'Priya', 'ECE', 92)
(3, 'Arjun', 'CSE', 78)
(4, 'Sneha', 'IT', 88)
(5, 'Kiran', 'CSE', 95)

Enter your question (or type exit): Show all students

Generated SQL:
SELECT * FROM students;

Query Result:
(1, 'Rahul', 'CSE', 85)
(2, 'Priya', 'ECE', 92)
(3, 'Arjun', 'CSE', 78)
(4, 'Sneha', 'IT', 88)
(5, 'Kiran', 'CSE', 95)

Enter your question (or type exit): Show students with marks greater than 85
```